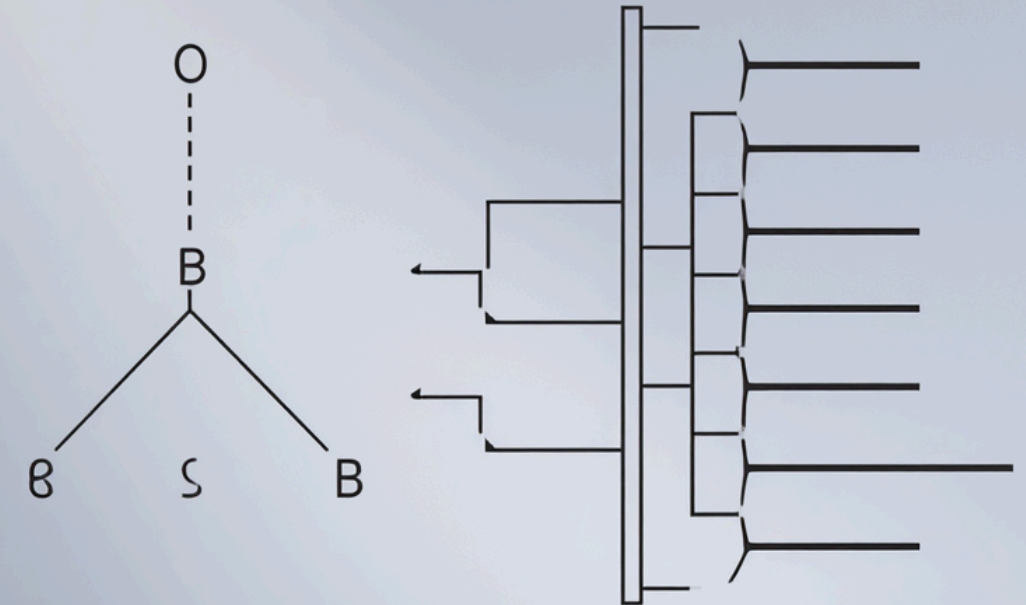


Flatten Binary Tree To LinkedList



GABRIEL, FABETA, KAYKY E ANTONIO



O Problema

Dada a raiz de uma árvore binária, achate-a em uma lista encadeada.

A lista deve usar a mesma classe `TreeNode`, em que o ponteiro `right` funciona como o `next` da lista e o ponteiro `left` é sempre nulo.

A ordem dos nós na lista deve ser a mesma da travessia em pré-ordem da árvore.

Restrições

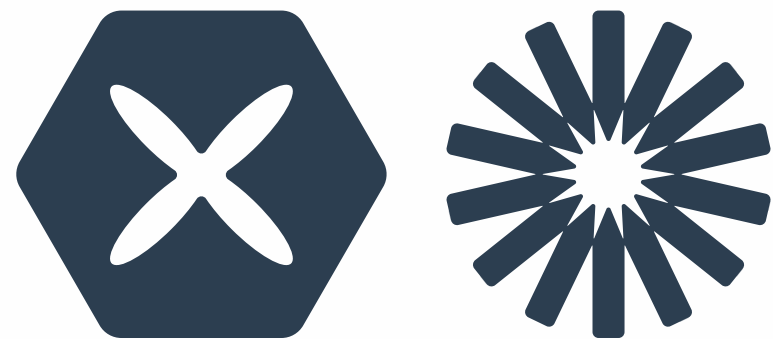
O número de nós da árvore está entre 0 e 2000.

Cada valor está entre -100 e 100.

Acompanhamento

Você consegue achatar a árvore in place, usando apenas espaço extra constante?

Por que achatar uma árvore?



Hierarquia não cabe em um fio

Toda estrutura em árvore precisa virar sequência quando sai da memória: para ser salva em disco, transmitida pela rede ou desenhada linha a linha na tela.

A pré-ordem preserva a estrutura

Ler a raiz, depois a esquerda, depois a direita é a ordem que permite reconstruir a árvore original depois.

In place economiza memória

Reaproveitar os ponteiros existentes evita alocar uma segunda estrutura do tamanho da entrada.

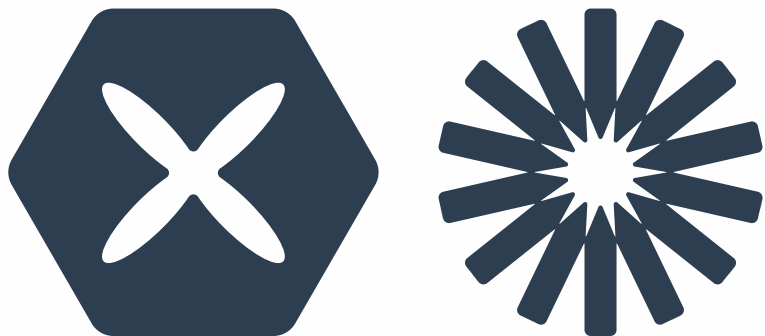
Onde isso aparece na prática

Listas virtualizadas na interface

Explorador de arquivos, comentários aninhados, tabelas agrupadas: a tela só desenha uma lista plana, e a pré-ordem dá exatamente a ordem que o usuário vê.

Serialização de HTML, XML e JSON

Salvar ou transmitir um documento exige linearizá-lo. O DOM é gravado em pré-ordem, e é assim que o navegador o reconstrói do outro lado.



Compiladores e árvores sintáticas

Achatar a AST em pré-ordem é o passo que transforma o código em uma sequência linear de instruções.

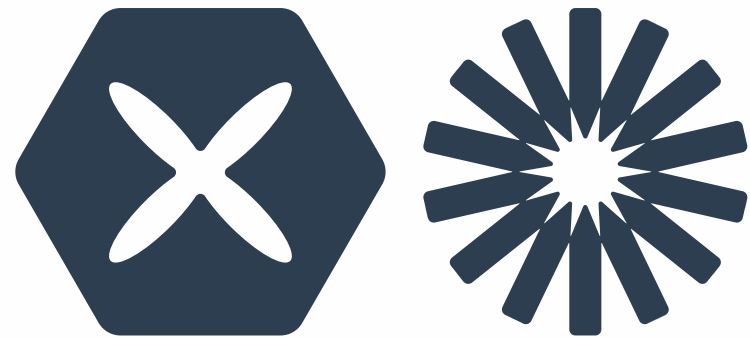


A abordagem ingênua apresenta várias limitações:

- **Complexidade:** Consome memória extra proporcional ao número de nós.
- **Desempenho:** Reencadeia após a travessia, destruindo ponteiros existentes.
- **Recursão:** A versão reversa ainda utiliza pilha proporcional à altura da árvore.

Esses fatores tornam a solução ineficiente para grandes árvores.

O Insight Central



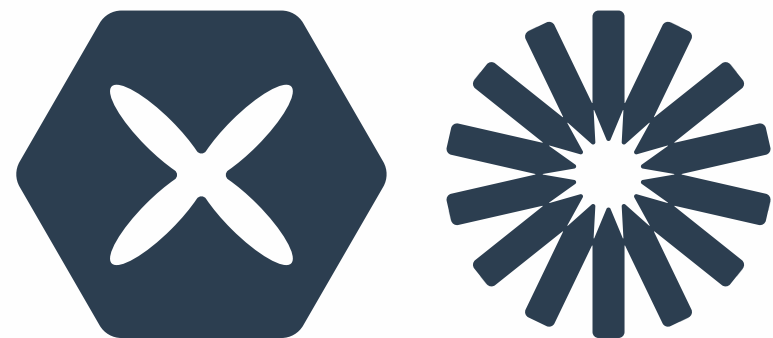
A Subárvore Esquerda e a Costura

A subárvore esquerda deve ser inserida entre o nó atual e a subárvore direita, conectando o último nó à subárvore direita.

Eliminação de Recursão e Memória

Essa abordagem evita o uso de recursão, permitindo uma travessia eficiente sem pilha, economizando memória e tornando o algoritmo mais eficiente.

Algoritmo Passo a Passo



Estrutura da Travessia Morris

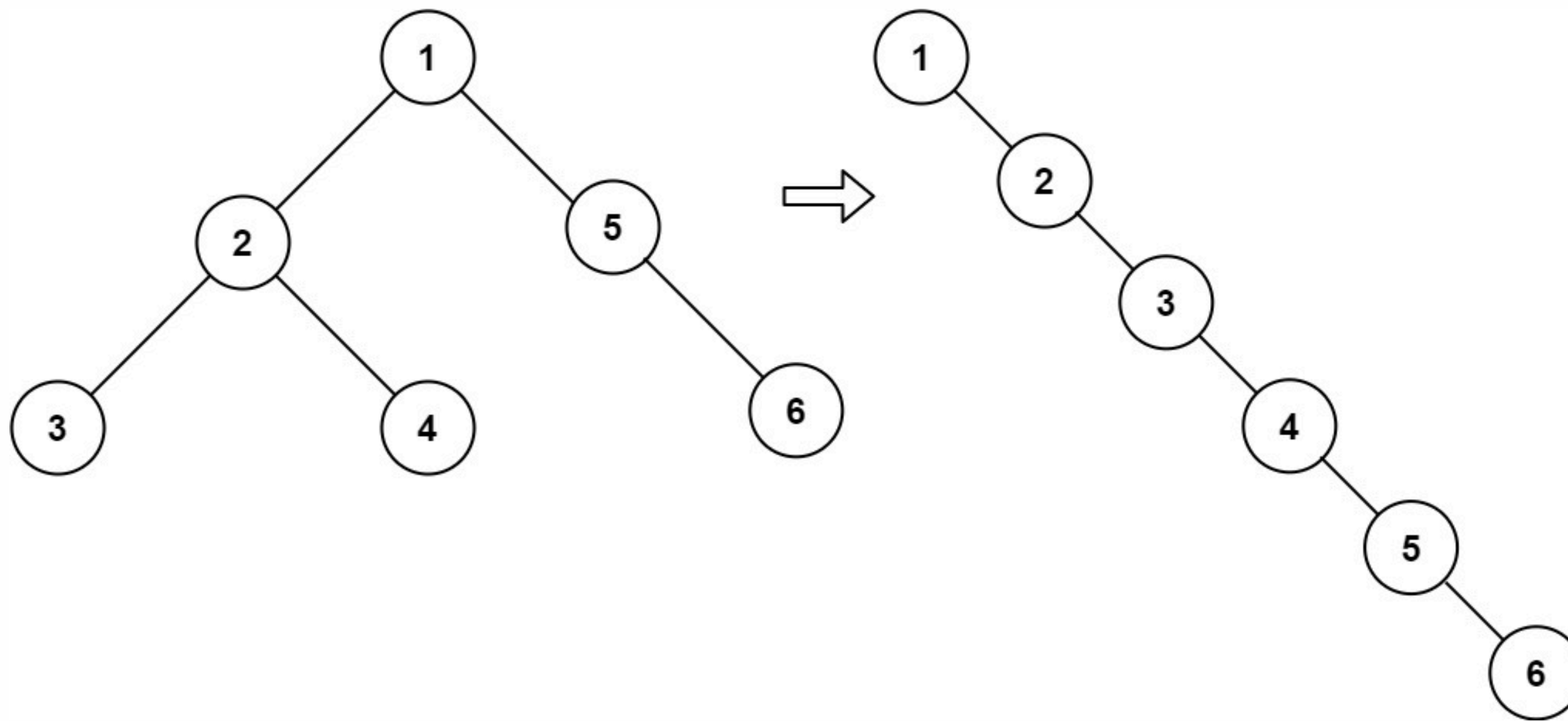
O algoritmo utiliza um ponteiro atual para percorrer a árvore, otimizando a travessia ao evitar a criação de estruturas auxiliares.

Mecanismo de Reassociação de Ponteiros

Ao chegar em um nó, conecta-se o último nó da subárvore esquerda ao filho direito do nó atual, mantendo a sequência.

Zerar o Ponteiro Esquerdo

É crucial zerar o ponteiro esquerdo após mover a subárvore, evitando referências desnecessárias e garantindo a integridade da lista encadeada.



Exemplos:

Input: root = [1,2,5,3,4,null,6]

Output: [1,null,2,null,3,null,4,null,5,null,6]

Input: root = []

Output: []

Input: root = [0]

Output: [0]

A ideia

Na pré-ordem, a subárvore direita vem logo depois do último nó da subárvore esquerda: o nó mais à direita dela, chamado predecessor.

Então basta religar ponteiros e descer pela direita. Sem recursão e sem pilha, em tempo linear e espaço constante.

Complexidade:

A complexidade de tempo desse código é $O(N)$ e a complexidade de espaço é $O(1)$, onde N é o número total de nós na árvore binária.

```
flatten.c

struct TreeNode *atual = root;

while (atual != NULL) {
    if (atual->left != NULL) {
        struct TreeNode *pred = atual->left;
        while (pred->right != NULL) {
            pred = pred->right;
        }
        pred->right = atual->right;
        atual->right = atual->left;
        atual->left = NULL;
    }
    atual = atual->right;
}
```

Quando não há o que religar

Sem filho esquerdo não há nada a religar: o próximo nó da pré-ordem já é o filho da direita.

Esse é o único teste do algoritmo. Se ele falha, o nó já está no formato final e basta avançar.

```
flatten.c  
  
if (atual->left != NULL) {  
    /* religa os ponteiros */  
}
```

1. Achar o predecessor

Desce uma vez pela esquerda e depois só pela direita, até o fim.

O nó onde essa descida para é o último da pré-ordem da subárvore esquerda: exatamente onde a subárvore direita precisa ser emendada.

```
flatten.c  
  
struct TreeNode *pred = atual->left;  
while (pred->right != NULL) {  
    pred = pred->right;  
}
```

2. Pendurar a subárvore direita

A subárvore direita passa a pendurar no predecessor, que agora é o fim da sequência da esquerda.

É seguro porque o predecessor sempre tem right igual a NULL: foi exatamente assim que a busca parou nele. Nada se perde.



```
flatten.c  
  
pred->right = atual->right;
```

3. A esquerda vira a nova direita

A subárvore esquerda, já com a direita colada no fim, passa a ser a direita do nó atual.

A ordem resultante é exatamente a pré-ordem: o nó, depois a subárvore esquerda, depois a direita.

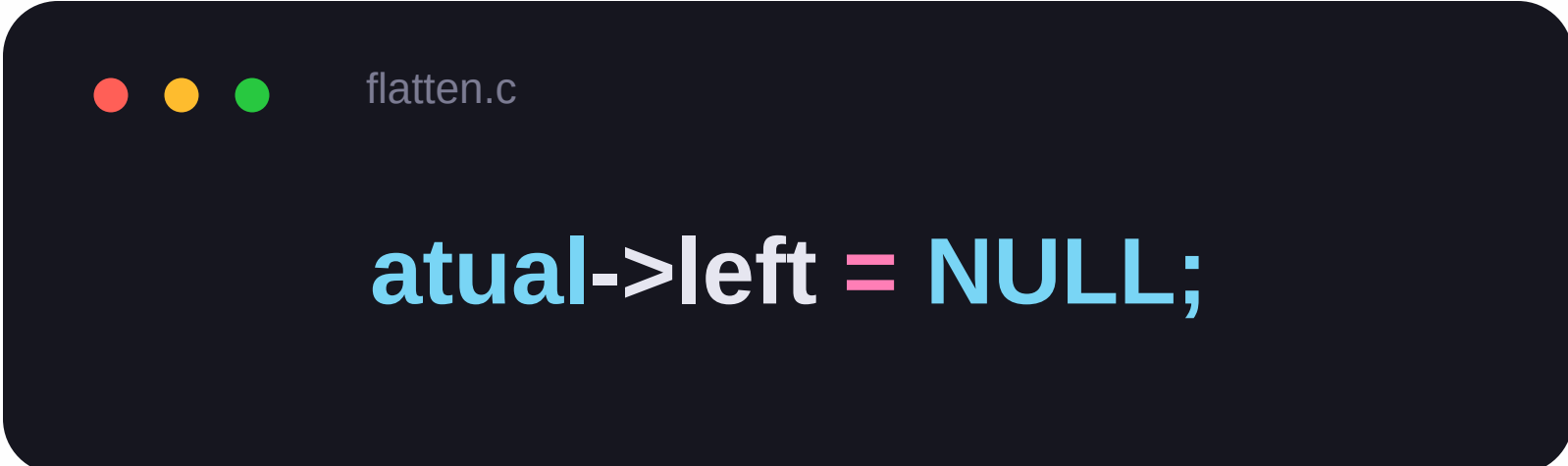


```
flatten.c  
  
atual->right = atual->left;
```

4. Zerar o ponteiro da esquerda

Exigência do enunciado: todo left vira NULL.

Sem isso o resultado continua sendo uma árvore, e não uma lista encadeada. É o detalhe que mais reprova submissão no LeetCode.



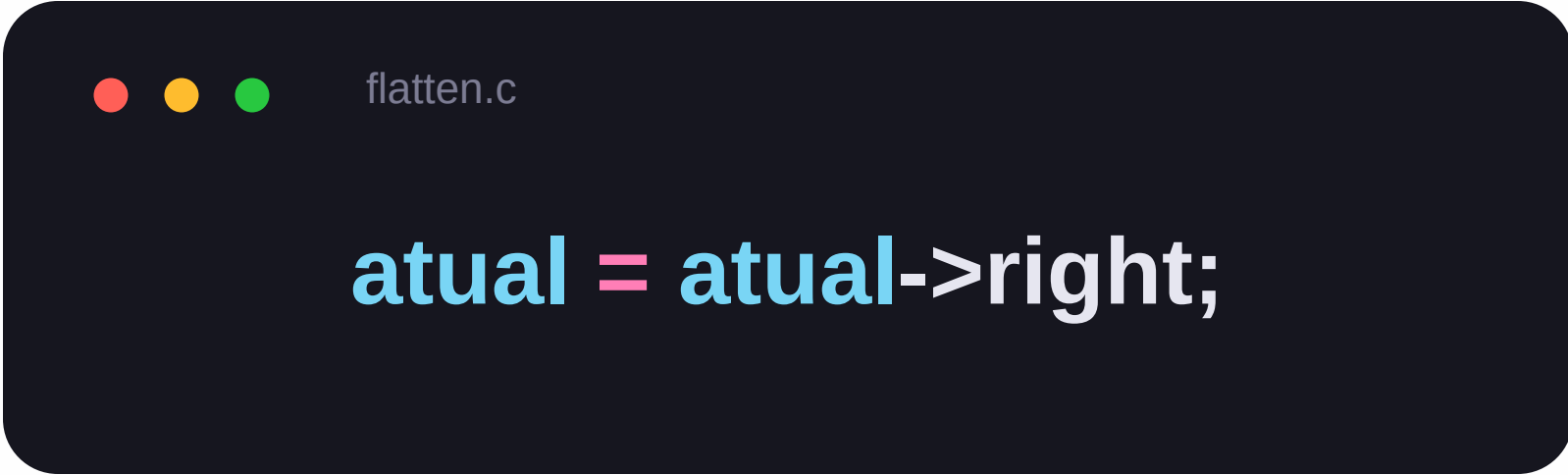
```
flatten.c  
  
atual->left = NULL;
```

Avançar

Não precisamos voltar: o que ficou para trás já está no formato final.

E a subárvore esquerda que acabou de ser movida será achatada quando o ponteiro atual passar por dentro dela.

Cada aresta é percorrida no máximo duas vezes: tempo linear, espaço constante.



```
flatten.c  
  
atual = atual->right;
```

Link Para o Código:

[**https://donthpad.com/flattenbinarytree**](https://donthpad.com/flattenbinarytree)

[**https://leetcode.com/problems/flatten-binary-tree-to-linked-list/**](https://leetcode.com/problems/flatten-binary-tree-to-linked-list/)



"A simplicidade é o último grau
de sofisticação."

– Leonardo da Vinci